OPERATING SYSTEMS

Building PES-VCS – A Version Control System from Scratch

Orange Problem Solving (Unit 4)

Name : SUMAN DIDDIMANI
SRN : PES1UG24AM409
Section : 4G

Phase 1: Object Storage Foundation

Screenshot 1A: Output of showing all tests passing

```
suman-diddimani@suman-diddimani-pc:~/PES1UG24AM409-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
suman-diddimani@suman-diddimani-pc:~/PES1UG24AM409-pes-vcs$
```

Screenshot 1B:Showing the sharded directory structure.

```
suman-diddimani@suman-diddimani-pc:~/PES1UG24AM409-pes-vcs$ find .pes/objects -type f
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
suman-diddimani@suman-diddimani-pc:~/PES1UG24AM409-pes-vcs$
```

Phase 2: Tree Objects

Screenshot 2A: Output of showing all tests passing.

```
suman-diddimani@suman-diddimani-pc:~/os-u4-orange-problem$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
suman-diddimani@suman-diddimani-pc:~/os-u4-orange-problem$
```

Screenshot 2B: Pick a tree object from `find .pes/objects -type f` and run `xxd .pes/objects/XX/YYY... | head -20` to show the raw binary format.

```
suman-diddimani@suman-diddimani-pc:~/os-u4-orange-problem$ find .pes/objects -type f
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
suman-diddimani@suman-diddimani-pc:~/os-u4-orange-problem$ xxd .pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf | head -20
00000000: 626c 6f62 2031 3500 5465 7374 2069 6e74  blob 15.Test int
00000010: 6567 7269 5879 0a                        egriXy.
suman-diddimani@suman-diddimani-pc:~/os-u4-orange-problem$
```

Phase 3: The Index (Staging Area)

Screenshot 3A: Run `./pes init`, `./pes add file1.txt file2.txt`, `./pes status` — show the output.

```
suman-diddimani@suman-diddimani-pc:~/PES1UG24AM409-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
suman-diddimani@suman-diddimani-pc:~/PES1UG24AM409-pes-vcs$ ./pes add file1.txt file2.txt
suman-diddimani@suman-diddimani-pc:~/PES1UG24AM409-pes-vcs$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  commit.c
  untracked:  object.c
  untracked:  objet.c
  untracked:  test_tree
  untracked:  Makefile
  untracked:  commit.h
  untracked:  index.h
  untracked:  README.md
  untracked:  test_sequence.sh
  untracked:  .gitignore
  untracked:  index.c
  untracked:  test_tree.c
  untracked:  tree.c
  untracked:  pes.h
  untracked:  tree.h
  untracked:  test_objects.c
  untracked:  pes.c
  untracked:  test_objects
  untracked:  tree.c.save

suman-diddimani@suman-diddimani-pc:~/PES1UG24AM409-pes-vcs$
```

Screenshot 3B: `cat .pes/index` showing the text-format index with your entries.

```
suman-diddimani@suman-diddimani-pc:~/PES1UG24AM409-pes-vcs$ cat .pes/index
100664 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776343553 6 file1.txt
100664 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776343557 6 file2.txt
suman-diddimani@suman-diddimani-pc:~/PES1UG24AM409-pes-vcs$
```

Phase 4: Commits and History

Screenshot 4A: Output of `./pes log` showing three commits with hashes, authors, timestamps, and messages.

```
suman-diddimani@suman-diddimani-pc:~/PES1UG24AM409-pes-vcs$ ./pes log
commit 70ceb27837ab7aa77cd2674684889a8ded1ed8ae089a761c98deb7f8128913da
Author: PES User <pes@localhost>
Date:   1776348481

    Add farewell

commit c0692f4186fe06590803eeb411168ac8b2eba8dc6dc11059476d91e555ce9b19
Author: PES User <pes@localhost>
Date:   1776348468

    Add worldell

commit 11a426a0c18116d2d933edee550d8a2f30a142754d0ca9048a9e2d054e7cca3e
Author: PES User <pes@localhost>
Date:   1776348451

    Initial committial commiQ
```

Screenshot 4B: `find .pes -type f | sort` **showing object store growth after three commits.**

```
suman-diddimani@suman-diddimani-pc:~/PES1UG24AM409-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/11/a426a0c18116d2d933edee550d8a2f30a142754d0ca9048a9e2d054e7cca3e
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/6e/f19b41225c5369f1c104d45d8d85efa9b057b53b14b4b9b939dd74decc5321
.pes/objects/70/ceb27837ab7aa77cd2674684889a8ded1ed8ae089a761c98deb7f8128913da
.pes/objects/c0/692f4186fe06590803eeb411168ac8b2eba8dc6dc11059476d91e555ce9b19
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main
suman-diddimani@suman-diddimani-pc:~/PES1UG24AM409-pes-vcs$
```

Screenshot 4C: `cat .pes/refs/heads/main` and `cat .pes/HEAD` showing the reference chain.

```
suman-diddimani@suman-diddimani-pc:~/PES1UG24AM409-pes-vcs$ cat .pes/refs/heads/main
70ceb27837ab7aa77cd2674684889a8ded1ed8ae089a761c98deb7f8128913da
suman-diddimani@suman-diddimani-pc:~/PES1UG24AM409-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
suman-diddimani@suman-diddimani-pc:~/PES1UG24AM409-pes-vcs$
```

Phase 5 — Branching and Checkout

Q5.1: A branch in Git is just a file in `.git/refs/heads/` containing a commit hash. Creating a branch is creating a file. Given this, how would you implement `pes checkout <branch>` — what files need to change in `.pes/`, and what must happen to the working directory? What makes this operation complex?

Ans : In the PES system, a branch is represented as a file in .pes/refs/heads/ that contains the hash of the latest commit of that branch. Implementing pes checkout <branch> would involve two main changes: updating repository metadata and updating the working directory.

First, the repository metadata must change. The HEAD file inside .pes/ should be updated to point to the selected branch. For example, if the user runs pes checkout feature, the HEAD file should contain ref: refs/heads/feature. This ensures that future commits advance the selected branch.

Next, the working directory must be updated to match the snapshot represented by the commit that the branch points to. The system would read the commit hash from .pes/refs/heads/<branch>, load the corresponding commit object from the object store, retrieve its tree object, and reconstruct the directory structure in the working directory. Files that do not exist in the target tree must be deleted, and files in the tree must be recreated from blob objects.

The complexity arises because the checkout operation must carefully update files in the working directory without overwriting user changes. The system must compare the current working directory, the index, and the target commit's tree to ensure that switching branches will not destroy uncommitted work. Handling nested directories and file deletions also increases the complexity of the operation.

Q5.2: When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store.

Ans:When switching branches, the system must ensure that uncommitted changes are not overwritten. A “dirty working directory” occurs when a tracked file has been modified in the working directory but those changes have not been staged or committed.

This conflict can be detected using the index and the object store. The index stores the hash of the staged version of each file. The algorithm would proceed as follows:

1. For every tracked file in the index, compute the current file’s metadata or content hash in the working directory.
2. Compare the working directory file with the version recorded in the index.
3. If they differ, the file has uncommitted modifications.
4. Then compare the same file path in the target branch’s tree.
5. If the file differs between the target branch and the current branch, switching branches would overwrite the user's changes.

If both conditions are true (modified locally and different in the target branch), checkout must refuse and display an error message. This prevents accidental loss of uncommitted changes.

Q5.3: "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?

Ans:A detached HEAD state occurs when the HEAD file contains a direct commit hash instead of a reference to a branch. In this state, new commits are still created normally, but they are not attached to any branch. As a result, no branch reference will point to those commits.

If the user continues committing in this state, the commits form a chain that is reachable only through the temporary HEAD pointer. If the user later switches branches, those commits may become unreachable and eventually be removed by garbage collection.

A user can recover commits made in detached HEAD state by creating a branch that points to the current commit. For example, running pes branch recover would create a new branch reference pointing to the current commit hash. This restores a

permanent reference to the commit history and prevents the commits from becoming unreachable.

Phase 6 — Garbage Collection and Space Reclamation

Q6.1: Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to (directly or transitively). Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.

Ans:Over time, the object store accumulates objects that are no longer reachable from any branch. To reclaim space, the system must identify objects that are not reachable from any reference.

The algorithm would start by identifying all root references, which include the commits pointed to by branch heads in .pes/refs/heads/ and possibly the HEAD pointer. From each root commit, the system performs a graph traversal. For each commit visited, the algorithm marks it as reachable, then follows references to its tree object and parent commit. From the tree object, it recursively visits all subtree objects and blob objects referenced by the tree.

All visited objects are stored in a data structure such as a hash set of object IDs. A hash set is efficient because it provides constant-time membership checks and prevents revisiting the same object multiple times.

After traversal, the system scans the .pes/objects/ directory and deletes any object whose hash does not appear in the reachable set.

For a repository with 100,000 commits and 50 branches, the number of commits visited would be at most about 100,000 because many branches share history. Each commit typically references one tree object, and each tree may reference several blobs. In practice, the traversal would likely visit a few hundred thousand objects total, but still remain manageable because objects are deduplicated and shared across commits.

Q6.2: Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?

Ans: Running garbage collection concurrently with a commit operation is dangerous because a race condition can occur between object creation and reference updates.

Consider this scenario: a commit operation writes a new blob and tree object to the object store but has not yet updated the branch reference to point to the new commit. At that moment, the garbage collector scans the repository and determines that the new objects are unreachable because no branch or commit references them yet. As a result, GC may delete those objects. When the commit operation finishes and updates the branch reference, it will point to objects that no longer exist, corrupting the repository.

Git avoids this problem by using several safety mechanisms. First, Git writes objects before updating references, ensuring that objects exist before they are referenced. Second, Git uses file locking to prevent concurrent updates to references. Third, Git’s garbage collector uses a grace period and does not immediately delete recently created objects. These mechanisms ensure that objects are not removed while another operation may still reference them.